PipedPeer: Transparent Distribution of Unmodified Python Workloads over a Zero-Configuration Peer Network

Yateen Vaviya
Department of Computer Engineering
Sardar Patel Institute of Technology
Mumbai, Maharashtra, India

Vinayak Yadav
Department of Computer Engineering
Sardar Patel Institute of Technology
Mumbai, Maharashtra, India

Sahil Gupta
Department of Computer Engineering
Sardar Patel Institute of Technology
Mumbai, Maharashtra, India

***Abstract*—Distributed computing frameworks require a cluster that somebody has already configured, and they require the program to be rewritten against their API. Both costs are paid before any speedup is obtained, which puts the idle machines on a small team's network out of reach for ordinary work. We present PipedPeer, a system that runs an unmodified Python script on another machine on the network with no cluster configuration and nothing installed on any participant beyond a single binary. Two mechanisms make this possible. First, the script's environment is built as a content-addressed Nix closure against a pinned package set, so the same script yields an identical store path on every machine and on any day; that path is then used as a cluster-wide cache key, turning environment transfer from a per-job cost into a once-per-cluster cost. On a three-node cluster the first submission of a job took 12.00 s and subsequent submissions of the same environment took 0.65 s, an eighteenfold reduction. Second, the parallel primitives the script already uses are substituted at import time by a shim injected into the job's interpreter, and are distributed only when a cost model fed by a measured bandwidth probe finds that moving the data costs less than computing it locally. The model declines to distribute dense matrix multiplication, and the project treats that refusal as a tested property rather than a missing feature. Every remote path degrades to a working local one: a worker killed during a distributed parallel map left the run to complete with correct results.**

***Index Terms*—distributed computing, peer-to-peer, transparent interception, reproducible environments, content-addressed caching, cost-based scheduling, fault tolerance**

I. Introduction

The gap between running `python script.py` and running the same work on several machines is large in operational effort and often small in the computation involved. Apache Spark [1], Ray [2] and Dask [3] all presuppose a configured cluster with a head node, matching software on every worker, and a scheduler process. Remote execution over `ssh` avoids that setup but provides no environment management, no placement, no isolation and no fault tolerance.

Two distinct obstacles are usually treated separately. The *environment* obstacle is that a remote machine must be made to match the local one, which is the bulk of the work and the reason ad hoc remote execution does not scale past one machine. The *code* obstacle is that using more than one machine normally requires rewriting the program.

PipedPeer removes both. Its contributions are:

1) **Peer-to-peer closure distribution keyed by store path.** Content-addressed store paths and substitution are Nix's, not ours [5]; what we add is pushing a closure between peers rather than pulling from a cache, a two-predicate presence check that is correct both for per-node stores and for a shared store, and the use of that check to decide which peers are eligible to receive distributed work (Section III-B).
2) **Transparent interception under a measured cost model.** Parallel primitives are substituted at import time and distributed only when the arithmetic favours it, with link bandwidth measured rather than assumed (Sections III-C, IV).
3) **A shuffle-only aggregation path.** Hash partitioning on the key is standard [11]; the choice here is to use *only* that path and never a combiner, which keeps every aggregation exact in value, including user-defined ones, at the cost of always paying a full shuffle (Section III-D).
4) **Placement without a scheduler.** Hard filters followed by a bounded score, evaluated in the submitting client, with a three-phase lease providing atomic reservation and recovery through expiry (Section III-A).
5) **A correctness-preserving fallback on every distributed path**, structural for the parallel-map race and by exception handling elsewhere, so a failed or absent cluster changes the running time but never the result (Section V).

II. Related Work

Spark [1], Ray [2] and Dask [3] are mature and considerably more capable at scale than the system described here; they differ in requiring a configured cluster and, for Spark and to a lesser degree the others, a program written against their API.

Two of them address the same two problems we do, and the distinction is narrow enough to state precisely. Ray ships a drop-in `ray.util.multiprocessing.Pool` and a `joblib` backend, and its runtime environments ship and

cache per-job dependencies; Modin [10] provides distributed pandas behind the pandas API. Each requires the user to change an import line and to have a cluster already running. What is different here is that no line of the user's program changes at all, because substitution happens in `sitecustomize` before the first user statement executes, and that the decision to distribute is gated on a measured cost model rather than taken whenever the API is used. We are not aware of prior work that combines import-time substitution of unmodified code with a per-call cost gate. BOINC [4] pioneered volunteer computing but is a platform for specific projects, each with its own server infrastructure, rather than a general executor. Nix [5] contributes the content-addressed store on which our caching argument rests.

Work on offloading in edge computing addresses the same decision our cost model makes. Deng et al. [6] formulate offloading as a comparison of transmission against local computation time; Yong et al. [7] show partial offloading to beat all-or-nothing strategies, which is the regime our interception operates in. Learned policies [8] exceed what a deterministic rule achieves, at the cost of a training phase that a zero-configuration tool cannot assume. Zhao et al. [9] show topology-aware placement to outperform scalar scoring, a limitation our placement function shares and which we identify as future work.

On failure transparency, Bergström [12] surveys abstractions that hide distributed failure from application code; since a user script here contains no failure handling at all, this is the property Section V must provide. The Python Array API [13] shares our goal of portability without code change but inverts the mechanism, asking libraries to converge on one interface rather than substituting the implementations behind interfaces users already call.

III. System Design

Every machine runs one daemon; the command-line client is short-lived. The coordinator is a library linked into that client rather than a service, so there is no scheduler process, no head node and no master. A machine is a submitter and a worker simultaneously, and a new participant joins by running the binary.

A. Placement

Candidates are gathered from a local store of known peers, from UDP probes on the subnet, and optionally from a registry. Five hard filters run in order: architecture must match the closure, a GPU must exist if required, and free memory, cores and VRAM must each satisfy the request. Survivors are scored:

$$S = \Big(1 - \tfrac{u_{\text{cpu}}}{200} - \tfrac{u_{\text{mem}}}{200} - 0.05\, n_{\text{jobs}} + 0.10(\hat{c} - \tfrac{1}{2}) + 0.06(\hat{f} - \tfrac{1}{2}) + 0.10(\hat{m} - \tfrac{1}{2}) + 0.10(\hat{r} - \tfrac{1}{2})\Big) \cdot H \quad (1)$$

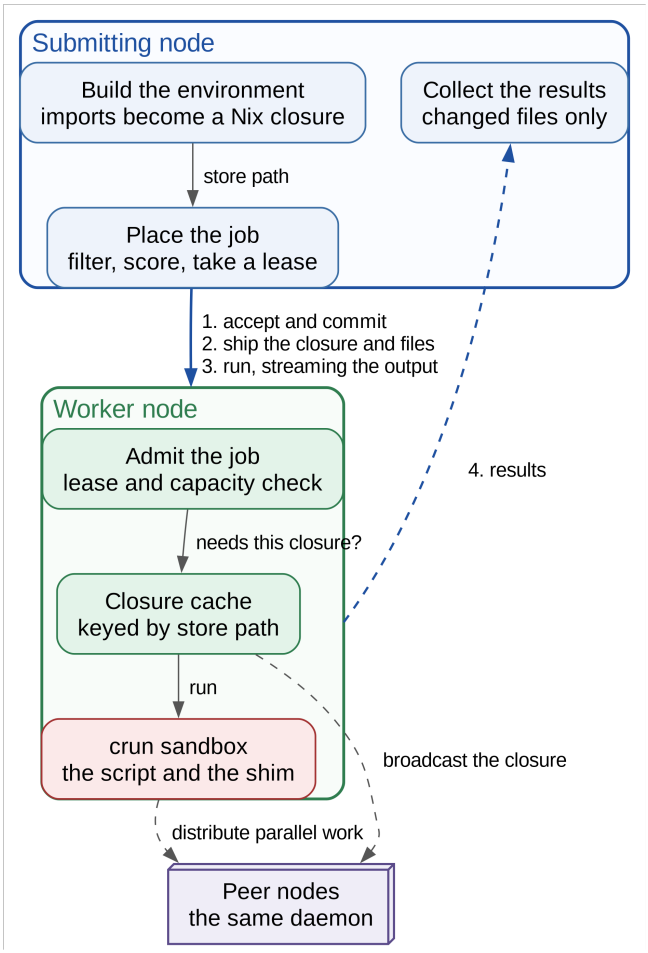

Fig. 1: System design. The submitting side builds the environment and places the job; the worker admits it, caches the closure and runs it sandboxed. Both roles are the same binary.

where u are utilisations in percent, n_{jobs} is the running job count, $\hat{c}, \hat{f}, \hat{m}, \hat{r}$ are core count, clock, free memory and free-core ratio normalised onto $[0, 1]$, and H is a health score. Two properties matter more than the coefficients. Load dominates hardware: the load terms can subtract 1.0 while the hardware terms move the score by at most 0.36, so a fast but busy machine loses to a slower idle one. And a missing metric normalises to 0.5, the neutral midpoint, so uneven telemetry across heterogeneous machines neither rewards nor penalises a node.

Work is reserved by a three-phase lease. The admission check and the lease insertion occur under one lock, so concurrent submitters cannot both claim the last slot, and the concurrency limit counts reserved and running leases together so a burst cannot slip through the window between the phases. Recovery needs no further mechanism: a submitter that dies stops renewing and its slot is reclaimed, while a worker that dies drops out of the healthy set and its job is rescheduled.

B. Environment Reproduction

The client scans the script's imports, resolves them to packages, generates a Nix flake and builds a closure. The flake pins an *exact* package-set revision rather than a branch, and this is the load-bearing decision of the entire design. A branch resolves at build time, so two machines building the same script on different days would produce different store paths; because the cache is keyed on the store path, the system would re-ship a large archive for every job. An exact pin makes the store path a stable, content-derived name for an environment across every machine and across time.

Before uploading, the client asks the target whether it already holds that path. Two distinct predicates are needed and both are implemented: whether the archive is cached, and whether the closure is materialised on disk. A deployment with a shared store volume satisfies the second without the first, and a per-node store satisfies the first before the second; either predicate alone is wrong for one of the two layouts. After an upload the worker pushes the closure to peers that lack it, which is what makes those peers eligible to receive distributed work.

C. Transparent Interception

A Python module embedded in the binary is appended to the job's workspace archive and placed on the interpreter's module path, so it is imported automatically before the user's first line. Nothing is installed on any node and the user's project on disk is never modified. Each library is patched on its first import rather than up front, since importing a large framework such as torch costs on the order of a second and charging that to jobs that never use it would violate the guarantee of Section V.

Substituted primitives are parallel maps, pandas grouping, joining and chunked reading, NumPy matrix multiplication and decompositions, `joblib` and scikit-learn parallelism, and data-parallel training.

D. Exact Aggregation Without Combiners

For grouping and joining, rows are bucketed by a hash of the key. Equal keys hash equally, so every key lands complete on exactly one node. Each node's aggregation is therefore a complete aggregation over the keys it holds, and the origin combines partials by concatenation alone. No combiner algebra is involved, and consequently *any* aggregation specification is exact in value, including user-defined reductions that admit no combiner. Systems that push combiners to the workers also fall back to a full shuffle for non-combinable aggregations, so this is not a stronger guarantee than theirs; it is the same guarantee reached by always taking the slower path, which suits a shim that cannot inspect the user's aggregation function. Exactness here is of values, not row order: concatenating buckets does not reproduce pandas' documented join ordering.

E. Data-Parallel Training Without a Socket Mesh

Standard data-parallel training establishes direct connections between every pair of ranks on ports of its own choosing, an assumption that holds in a data centre and often nowhere else. Instead, each rank posts its gradient to the lead node's ordinary daemon port and receives the full set once every rank has arrived; the daemon never interprets the payload and averaging happens in the shim. This trades bandwidth for requiring nothing beyond the port the rest of the system already uses.

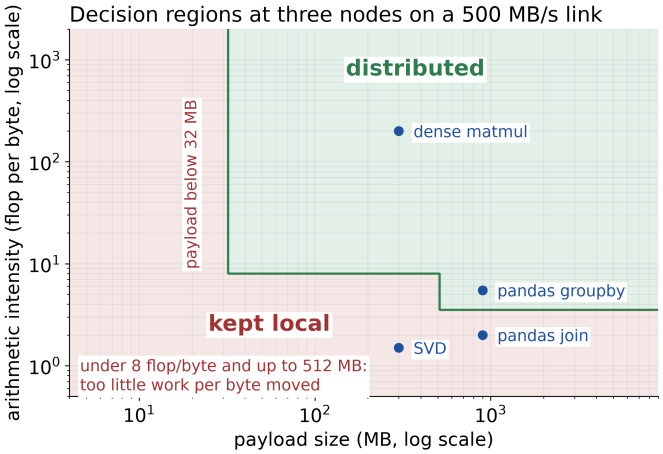

Fig. 2: Decision regions of Equation 2 at three nodes on a 500 MB/s link.

IV. The Cost Model

Interception is only worthwhile when the cluster wins, so every intercepted call is gated. Writing b for payload bytes, ϕ for arithmetic intensity in operations per byte, B for measured bandwidth, R for local throughput and K for node count, work is distributed only when

$$\frac{b\phi}{R} > \frac{b}{B} + \frac{b\phi}{RK} \times 1.3 \tag{2}$$

the factor 1.3 charging 30% overhead against ideal parallel speedup. Three guards precede the arithmetic: at least two nodes, a payload of at least 32 MB, and a carve-out rejecting low-intensity work below 512 MB. Bandwidth is measured by pushing 64 MB through the real dispatch path, cached for five minutes; if the probe fails the answer is to stay local.

A separately calibrated model governs dense linear algebra and produces a result worth emphasising: **matrix multiplication is never distributed**. Local BLAS sustains roughly 200 GFLOP/s, which no cluster transport beats at practical sizes. The project encodes this as a tested property, failing its demonstration suite if a matrix multiplication is ever observed leaving a node. Singular value decomposition, two orders of magnitude slower locally, does move.

V. Degradation Guarantee

Using the cluster must never change the result, and must not leave the caller worse off than a local run when the cluster is slow or absent. The timing half of that is bounded rather than absolute: Section VI-C measures the cost of leaving interception on at a few per cent of a short job, and only the

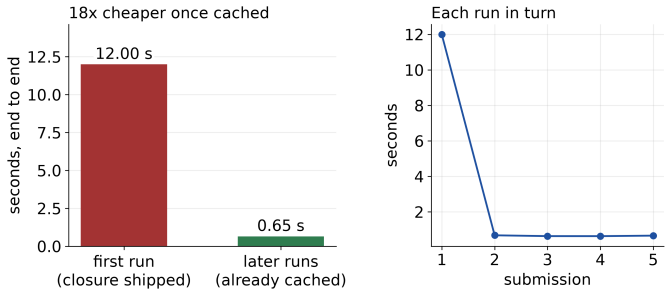

Fig. 3: Cold and warm submission of the same environment.

parallel-map path has a structural bound. Five layers apply. Chunks of a parallel map are dealt alternately to the local side and the cluster and fill a result table first-wins, with idle local cores re-running any chunk the cluster has not returned; the call returns only once the local side has covered every item, so a wholly unresponsive cluster degrades to a plain local pool. Beneath that, every intercepted call falls back to the original implementation on any exception, a failed peer is skipped in favour of the next, a dead worker process is respawned, and a failed job is rescheduled with exponential backoff.

VI. Evaluation

A. Method and its limits

Measurements were taken on one host with 20 logical cores running three worker daemons in containers. **This bed cannot measure multi-machine speedup**, because moving processor-bound work between containers on one host cannot make that host faster, and no speedup figure is claimed. It measures correctness, overhead, cache behaviour and fault tolerance, which are the properties the design argument rests on.

B. Environment cache

The same job submitted five times took 12.00 s on the first submission and a median of 0.65 s thereafter, an **eighteenfold reduction** (Fig. 3). The first submission ships the closure; the rest find it materialised and ship only the project files. This is the direct consequence of pinning the package set so that a store path names an environment identically everywhere.

C. Cost of leaving interception on

Since interception is on by default, its cost when the model declines to distribute is what most users pay. On a workload the model declines, a whole remote job took 0.93 s against 0.85 s with interception disabled, an overhead of **9%**.

Interpreter startup for a job that imports nothing heavy is a separate matter. Eagerly patching libraries cost $21.60\times$ the unshimmed startup, against a budget of $3\times$ the project sets itself; deferring each library's patching to its first import brings this to $1.20\times$.

TABLE I: PipedPeer against existing systems. The lower block is where it is the weaker system.

Property	Ours	Spark	Ray	Dask	ssh
What it does differently					
Runs unmodified Python	yes	no	part.	part.	yes
No cluster to configure	yes	no	no	no	part.
No head or master node	yes	no	no	no	yes
Reproduces the environment	yes	part.	part.	part.	no
Sandboxes each job	part.	no	no	no	no
Survives losing a node	yes	yes	yes	yes	no
Where it is behind					
Authenticated, encrypted	**no**	yes	part.	part.	yes
Works beyond one LAN	part.	yes	yes	yes	yes
Runtimes beyond Python	**no**	yes	part.	no	yes
Proven in production	**no**	yes	yes	yes	yes

D. Correctness and fault tolerance

The suite passes 246 test cases with none failing. Distribution correctness is established by comparison against local execution: grouped aggregation is frame-equal to local pandas, the four join types are equal up to row order (a concatenation of shuffle buckets does not reproduce pandas' join ordering), and each rank's gradient in data-parallel training is exactly the mean across ranks.

Under fault injection, a twenty-item parallel map was distributed across the cluster and a worker was killed mid-computation. The run completed with correct results, the checksum matching the serial computation.

The upper block lists the criteria the system was designed to change, so it wins them by construction and they should be discounted accordingly. The lower block is the informative half. Ours is the only system in the table with no authentication and no transport encryption whatsoever, which confines it to an already-trusted network; it is also the only one limited to one language and one subnet, and the only one without a production track record. Spark, Ray and Dask are substantially more capable at scale. The claim is not that this system is better, but that it occupies a different point on the curve, trading reach, security and maturity for the removal of setup cost and code change.

E. Threats to validity

The single-host bed is the principal threat and the reason no speedup is claimed. Container workers share a processor, a memory system and the loopback interface, so both transfer costs and contention differ from a real network. The cache result is the least affected, since it turns on whether an archive is sent at all rather than on how fast it travels; the interception overhead figures are the most affected and should be read as lower bounds on the benefit and upper bounds on the relative overhead.

VII. Conclusion

PipedPeer runs unmodified Python on other machines with no cluster to configure, by reproducing environments as content-addressed closures whose store path serves as a cluster-wide cache key, and by substituting the parallel primitives a script already uses under a cost model driven by measured bandwidth. The measurements that carry the argument are an eighteenfold reduction in repeat submission cost and correct completion of a distributed computation after a worker was killed mid-run.

The part we consider most transferable is the discipline of the cost model. A system that distributed everything it could intercept would be slower than plain Python on most real workloads, because a modern BLAS or a warm page cache usually beats the network. Measuring rather than assuming, declining to distribute when local wins, and treating that refusal as a property to be tested is what makes always-on interception defensible.

Future work is led by authentication and an encrypted transport, without which the system is confined to a trusted network; then discovery beyond the subnet, locality- and topology-aware placement, and measurement across separate machines to obtain the speedup figures this evaluation deliberately does not claim.

References

[1] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, “Spark: Cluster computing with working sets,” in *Proc. 2nd USENIX Workshop on Hot Topics in Cloud Computing*, 2010.

[2] P. Moritz et al., “Ray: A distributed framework for emerging AI applications,” in *Proc. 13th USENIX OSDI*, 2018, pp. 561–577.

[3] M. Rocklin, “Dask: Parallel computation with blocked algorithms and task scheduling,” in *Proc. 14th Python in Science Conf.*, 2015, pp. 130–136.

[4] D. P. Anderson, “BOINC: A system for public-resource computing and storage,” in *Proc. 5th IEEE/ACM Workshop on Grid Computing*, 2004, pp. 4–10.

[5] E. Dolstra, M. de Jonge, and E. Visser, “Nix: A safe and policy-free system for software deployment,” in *Proc. 18th LISA*, 2004, pp. 79–92.

[6] Y. Deng, Z. Chen, X. Chen, and Y. Fang, “Task offloading in multi-hop relay-aided multi-access edge computing,” *IEEE Trans. Veh. Technol.*, vol. 72, no. 1, pp. 1372–1376, 2023.

[7] D. Yong, R. Liu, X. Jia, and Y. Gu, “Joint optimization of multi-user partial offloading strategy and resource allocation in D2D-enabled MEC,” *Sensors*, vol. 23, no. 5, 2023.

[8] H. He, X. Yang, X. Mi, H. Shen, and X. Liao, “Multi-agent deep reinforcement learning based dynamic task offloading in a D2D mobile-edge computing network,” *Sensors*, vol. 24, no. 16, 2024.

[9] Z. Zhao, J. Perazzone, G. Verma, and S. Segarra, “Congestion-aware distributed task offloading in wireless multi-hop networks using graph neural networks,” arXiv:2312.02471, 2023.

[10] D. Petersohn et al., “Towards scalable dataframe systems,” *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 2033–2046, 2020.

[11] J. Dean and S. Ghemawat, “MapReduce: Simplified data processing on large clusters,” in *Proc. 6th USENIX OSDI*, 2004, pp. 137–150.

[12] A. Bergström, “Programming models for failure-transparent distributed systems,” M.S. thesis, KTH Royal Institute of Technology, 2025.

[13] A. Meurer et al., “Python array API standard: Toward array interoperability in the scientific Python ecosystem,” in *Proc. SciPy*, 2023.